

Laporan Akhir Tugas Besar
Pemrograman Berorientasi Objek
MoviX



1. Aloysius Axel Adriano (103012400419)
2. Ghifari Nurwafi Yoga (103012400020)
3. Sava Arsyia Syandana (103012400043)
4. Daffa Dzaki Taufiqul Hakim (103012400107)
5. Muhammad Shaquille Shah Alam Murtopo (103012400225)

S1 Informatika
Fakultas Informatika
Universitas Telkom
2026

DAFTAR ISI

1. PENDAHULUAN.....	8
1.1 LATAR BELAKANG.....	8
1.2 TUJUAN.....	8
1.3 DESKRIPSI UMUM SISTEM (RUANG LINGKUP).....	8
1.4 METODOLOGI PENGEMBANGAN SISTEM.....	9
1.4.1 ANALISIS KEBUTUHAN.....	9
1.4.2 IMPLEMENTASI SISTEM.....	9
1.4.3 PENGUJIAN SISTEM.....	9
2 DASAR TEORI.....	9
2.1 PEMROGRAMAN BERORIENTASI OBJEK.....	9
2.2 UNIFIED MODELING LANGUAGE (UML).....	10
2.3 JAVA.....	10
2.4 MYSQL.....	10
2.5 SERVLET/JSP/SPRINGBOOT.....	10
2.6 POLA DESAIN DATA ACCESS OBJECT (DAO).....	11
2.7 Arsitektur Model-View-Controller (MVC).....	11
2.8 Java Database Connectivity (JDBC).....	11
3 ANALISIS DAN IMPLEMENTASI SISTEM.....	12
3.1 ANALISIS KEBUTUHAN SISTEM.....	12
3.1.1 KEBUTUHAN FUNGSIONAL (FITUR).....	12
3.1.2 KEBUTUHAN NON-FUNGSIONAL.....	13
3.2 IMPLEMENTASI BASIS DATA.....	14
3.2.1 STRUKTUR BASIS DATA.....	15
3.3 IMPLEMENTASI UML.....	20
3.3.1 CLASS DIAGRAM.....	20
3.3.2 DESKRIPSI KELAS.....	20
3.4 IMPLEMENTASI ANTARMUKA PENGGUNA (USER INTERFACE).....	22
3.4.1 HALAMAN LOGIN.....	22
3.4.2 HALAMAN DASHBOARD ADMIN.....	23
3.4.3 HALAMAN KATALOG FILM (BERANDA PELANGGAN).....	24
3.4.4 Halaman Pemilihan Kursi.....	24
3.4.5 Halaman Checkout dan Pembayaran.....	25
4 PENGUJIAN SISTEM.....	25
4.1 TUJUAN PENGUJIAN.....	25
4.2 LINGKUNGAN PENGUJIAN.....	25
4.2.1 PERANGKAT LUNAK PENGUJIAN.....	25
4.2.2 PERANGKAT KERAS PENGUJIAN.....	25
4.3 METODE PENGUJIAN.....	26
4.3.1 (BLACK BOX/WHITE BOX) TESTING.....	26
4.3.2 SKENARIO PENGUJIAN.....	26

5 HASIL UJI.....	27
5.1 FITUR LOGIN.....	27
5.1.1 KASUS UJI.....	27
5.1.2 HASIL.....	27
5.2 FITUR LOGOUT.....	27
5.2.1 KASUS UJI.....	27
5.2.2 HASIL.....	27
5.3 FITUR KATALOG FILM INTERAKTIF (ADMIN).....	28
5.3.1 KASUS UJI.....	28
5.3.2 HASIL.....	28
5.4 FITUR PEMILIHAN JADWAL DAN STUDIO.....	29
5.4.1 KASUS UJI.....	29
5.4.2 HASIL.....	29
5.5 FITUR PEMILIHAN KURSI DINAMIS.....	30
5.5.1 KASUS UJI.....	30
5.5.2 HASIL.....	30
5.6 FITUR SISTEM CHECKOUT DAN KODE PROMO.....	31
5.6.1 KASUS UJI.....	31
5.6.2 HASIL.....	32
6 PENUTUP.....	33
6.1 KESIMPULAN.....	33
6.2 SARAN.....	33
DAFTAR PUSTAKA.....	34
Lampiran.....	35

DAFTAR TABEL

Tabel 1 Tabel Entitas User.....	9
Tabel 2 Tabel Film.....	9
Tabel 3 Tabel Studio.....	9
Tabel 4 Tabel Jadwal Tayang.....	9
Tabel 5 Tabel Promo.....	9
Tabel 6 Tabel Transaksi.....	9
Tabel 7 Identifikasi Pengujian.....	10
Tabel 8 Kasus Uji Fitur Login.....	10
Tabel 9 Hasil Uji Fitur Login.....	10
Tabel 10 Kasus Uji Fitur Logout.....	11
Tabel 11 Hasil Uji Fitur Logout.....	11
Tabel 12 Kasus Uji Fitur Katalog Film Interaktif.....	11
Tabel 13 Hasil Uji Fitur Katalog Film Interaktif.....	11
Tabel 14 Kasus Uji Fitur Pemilihan Jadwal dan Studio.....	11
Tabel 15 Hasil Uji Fitur Pemilihan Jadwal dan Studio.....	11
Tabel 16 Kasus Uji Fitur Pemilihan Kursi Dinamis.....	11
Tabel 17 Hasil Uji Fitur Pemilihan Kursi Dinamis.....	11
Tabel 18 Kasus Uji Fitur Sistem Checkout dan Kode Promo.....	12
Tabel 19 Hasil Uji Fitur Sistem Checkout dan Kode Promo.....	12
Tabel 20 Pembagian Tugas.....	14

DAFTAR GAMBAR

Gambar 1 ERD Sistem.....	14
Gambar 2 Class Diagram.....	20
Gambar 3 Halaman Registrasi.....	22
Gambar 4 Halaman Login.....	22
Gambar 5 Halaman Dashboard Admin.....	23
Gambar 6 Halaman Katalog Film.....	24
Gambar 7 Halaman Pemilihan Kursi.....	24
Gambar 8 Halaman Checkout dan Pembayaran.....	25
Gambar 9 Screenshot Sistem.....	36

1. Pendahuluan

1.1 Latar Belakang

Perkembangan teknologi informasi saat ini telah mendorong proses digitalisasi di berbagai sektor industri, termasuk industri hiburan seperti bioskop. Sistem pemesanan tiket secara konvensional seringkali menimbulkan antrean panjang dan kurang efisien bagi pelanggan maupun pihak pengelola. Oleh karena itu, dikembangkanlah MoviX, sebuah aplikasi pemesanan tiket bioskop digital.

Pengembangan aplikasi ini sekaligus menjadi sarana implementasi konsep Pemrograman Berorientasi Objek (PBO) secara nyata. Dengan menggunakan bahasa pemrograman Java dan arsitektur yang terstruktur, aplikasi ini dirancang agar memiliki *source code* yang modular, mudah dikelola (*maintainable*), dan dapat dikembangkan lebih lanjut. Pola perancangan seperti *Model-View-Controller* (MVC) serta pilar-pilar utama OOP (*Encapsulation, Inheritance, Polymorphism*, dan *Abstraction*) diterapkan untuk menyelesaikan masalah kompleksitas pengelolaan data tiket, jadwal tayang, dan transaksi pengguna.

1.2 Tujuan

Adapun tujuan dari pengembangan sistem pada Tugas Besar ini adalah sebagai berikut:

1. Mengimplementasikan pilar-pilar utama Pemrograman Berorientasi Objek (PBO) dalam sebuah studi kasus perangkat lunak skala menengah.
2. Membangun sistem aplikasi MoviX untuk memudahkan pelanggan dalam melakukan pemesanan tiket bioskop, pemilihan kursi, serta penggunaan kode promo secara mandiri.
3. Membangun antarmuka dan sistem manajemen basis data (*database*) untuk memudahkan Admin dalam mengelola master data bioskop, seperti data film, studio, jadwal tayang, dan promo diskon.

1.3 Deskripsi Umum Sistem (Ruang Lingkup)

Aplikasi pemesanan tiket bioskop "MoviX" adalah sistem berbasis web/desktop yang membagi hak akses penggunanya menjadi dua peran (*role*) utama, yaitu **Admin** dan **Pelanggan**.

- **Admin:** Memiliki kewenangan penuh terhadap pengelolaan data inti sistem (*Master Data*). Admin dapat menambah, membaca, mengubah, dan menghapus (CRUD) entitas Film, Studio, Jadwal Tayang, dan Promo.

- **Pelanggan:** Memiliki hak akses transaksional. Pelanggan dapat melihat katalog film yang sedang tayang, memilih jadwal berdasarkan studio yang tersedia, memilih kursi spesifik secara presisi, memasukkan kode promo (baik berupa diskon persentase maupun nominal), dan menyelesaikan proses *checkout* atau pemesanan tiket.

Sistem ini didukung oleh basis data relasional (MySQL) untuk menjamin integritas data transaksi dan ketersediaan kursi secara *real-time*.

1.4 Metodologi Pengembangan Sistem

1.4.1 Analisis Kebutuhan

Melakukan pengumpulan dan analisis terhadap kebutuhan fungsional (fitur-fitur utama Admin dan Pelanggan) serta kebutuhan non-fungsional sistem. Pada tahap ini, dilakukan perancangan relasi basis data (ERD) dan pemodelan sistem menggunakan *Unified Modeling Language* (UML) seperti *Class Diagram*.

1.4.2 Implementasi Sistem

Mentransformasikan rancangan desain menjadi kode program yang nyata. Tahap ini mencakup pembuatan antarmuka pengguna (*User Interface*), penulisan logika program menggunakan Java berorientasi objek (*Encapsulation, Inheritance, dll*), hingga penerapan pola *Data Access Object* (DAO) untuk menghubungkan aplikasi dengan basis data MySQL.

1.4.3 Pengujian Sistem

Melakukan pengujian (*testing*) terhadap sistem yang telah dibangun menggunakan metode *Black Box Testing*. Pengujian ini difokuskan pada fungsionalitas antarmuka dan *input-output* perangkat lunak, seperti pengujian kelancaran fungsi *login/logout*, validasi ketersediaan kursi, dan akurasi kalkulasi potongan harga dari kode promo, guna memastikan sistem berjalan tanpa kesalahan teknis (*bug*).

2 Dasar Teori

2.1 Pemrograman Berorientasi Objek

Pemrograman Berorientasi Objek (PBO) atau *Object-Oriented Programming* (OOP) adalah sebuah paradigma pemrograman yang berpusat pada konsep "objek". Objek merupakan representasi dari entitas di dunia nyata yang di dalamnya menggabungkan struktur data (atribut) dan perilaku (metode). Terdapat empat pilar utama dalam PBO, yaitu:

1. **Encapsulation (Enkapsulasi):** Mekanisme untuk membungkus data (variabel) dan kode yang bertindak atas data (metode) bersama-sama sebagai satu kesatuan. Hal ini membatasi akses langsung ke beberapa komponen objek guna melindungi integritas data.

2. Inheritance (Pewarisan): Konsep di mana sebuah kelas dapat mewarisi atribut dan metode dari kelas lain (*superclass*), sehingga memfasilitasi penggunaan ulang kode (*code reusability*).
3. Polymorphism (Polimorfisme): Kemampuan entitas (seperti metode atau objek) untuk memiliki berbagai bentuk implementasi, memungkinkan suatu aksi dilakukan dengan cara yang berbeda berdasarkan objek yang memanggilnya.
4. Abstraction (Abstraksi): Proses menyembunyikan kerumitan detail implementasi dari pengguna dan hanya memaparkan fungsionalitas atau antarmuka esensialnya saja.

2.2 **Unified Modeling Language (UML)**

Unified Modeling Language (UML) adalah bahasa standar yang digunakan di industri perangkat lunak untuk memvisualisasikan, merancang, dan mendokumentasikan artefak dari sistem perangkat lunak. Salah satu diagram struktural yang paling penting dalam UML adalah *Class Diagram*. Diagram kelas mendeskripsikan struktur statis sebuah sistem dengan menampilkan kelas-kelas dalam sistem, atributnya, metodenya, serta hubungan antar-kelas tersebut (seperti asosiasi, agregasi, komposisi, dan pewarisan).

2.3 **Java**

Java adalah bahasa pemrograman tingkat tinggi berorientasi objek yang dikembangkan dengan semboyan "Write Once, Run Anywhere" (WORA). Ini berarti kode Java yang telah dikompilasi (menjadi bytecode) dapat dijalankan di berbagai platform tanpa perlu kompilasi ulang, berkat adanya Java Virtual Machine (JVM). Java menyediakan fitur keamanan bawaan, manajemen memori otomatis (Garbage Collection), serta dukungan library yang sangat luas, menjadikannya standar industri untuk pengembangan sistem backend berskala menengah hingga besar.

2.4 **MySQL**

MySQL adalah Relational Database Management System (RDBMS) berbasis open-source yang menggunakan Structured Query Language (SQL) untuk mengelola basis datanya. MySQL memungkinkan data untuk disimpan ke dalam tabel-tabel terpisah yang dapat dihubungkan (berelasi) satu sama lain menggunakan kunci utama (Primary Key) dan kunci tamu (Foreign Key). RDBMS ini andal dalam menangani penyimpanan, manipulasi (CRUD), dan pengambilan data secara persisten untuk menjamin keberlangsungan sistem transaksi pemesanan tiket.

2.5 **Servlet/JSP/Springboot**

Java Servlet adalah komponen program berbasis Java yang berjalan pada web server atau application server dan bertindak sebagai jembatan (controller) antara permintaan (HTTP request) dari web browser klien dengan basis data. Servlet membaca data yang dikirimkan klien (misalnya form pemesanan tiket), memproses logika sistem, dan mengembalikan respons

(HTTP response) ke klien. Dalam penerapannya, Servlet sering digabungkan dengan teknologi JavaServer Pages (JSP) yang berfungsi untuk menampilkan antarmuka web (HTML/CSS) secara dinamis yang diperkaya dengan data dari objek Java.

2.6 Pola Desain Data Access Object (DAO)

Data Access Object (DAO) adalah pola desain (design pattern) arsitektur perangkat lunak yang memisahkan antara logika antarmuka tingkat tinggi dengan logika akses basis data tingkat rendah. Melalui kelas DAO, semua interaksi langsung dengan database seperti query SQL, koneksi, serta operasi CRUD (Create, Read, Update, Delete) akan diabstraksi dan dipusatkan. Penggunaan DAO membuat struktur program menjadi lebih rapi, modular, dan mempermudah pemeliharaan sistem di masa mendatang tanpa perlu mengubah business logic dari aplikasi utama.

2.7 Arsitektur Model-View-Controller (MVC)

Model-View-Controller (MVC) adalah sebuah pola arsitektur perangkat lunak yang memisahkan aplikasi ke dalam tiga komponen logika utama:

1. Model: Mewakili struktur data dan logika bisnis aplikasi. Dalam sistem ini, Model direpresentasikan oleh kelas-kelas entitas (POJO) seperti Film, Studio, Jadwal Tayang, dan Transaksi.
2. View: Menangani antarmuka pengguna (UI) dan presentasi data. View bertanggung jawab menampilkan informasi kepada pengguna (Admin maupun Pelanggan) yang umumnya diimplementasikan menggunakan HTML/JSP.
3. Controller: Bertindak sebagai perantara antara Model dan View. Controller menerima request dari View, memprosesnya menggunakan aturan bisnis pada Model (atau melalui DAO), dan mengembalikan hasil pemrosesan kembali ke View. Pada sistem ini, Controller diimplementasikan menggunakan kelas-kelas Servlet seperti AuthServlet dan CheckoutServlet.

2.8 Java Database Connectivity (JDBC)

Java Database Connectivity (JDBC) adalah Application Programming Interface (API) bawaan dari Java yang mendefinisikan bagaimana sebuah program Java dapat mengakses, mengelola, dan mengeksekusi query ke dalam basis data relasional. JDBC menggunakan antarmuka seperti Connection, Statement, PreparedStatement, dan ResultSet. Dalam aplikasi MoviX, JDBC diimplementasikan pada kelas DbConnection untuk membangun jembatan koneksi sistem ke database MySQL, yang kemudian dimanfaatkan oleh kelas-kelas DAO untuk melakukan operasi pengolahan data.

3 Analisis dan Implementasi Sistem

3.1 Analisis Kebutuhan Sistem

Tahap analisis kebutuhan sistem bertujuan untuk mengidentifikasi fungsi-fungsi spesifik apa saja yang harus dapat dilakukan oleh sistem (fungsional) serta batasan spesifikasi teknis lingkungan sistem untuk berjalan dengan optimal (non-fungsional).

3.1.1 Kebutuhan Fungsional (Fitur)

Kebutuhan fungsional mendefinisikan kemampuan aplikasi MoviX dalam menerima input, memproses data, dan menghasilkan output sesuai dengan *role* masing-masing pengguna (Admin dan Pelanggan).

3.1.1.1. Fitur Autentikasi Pengguna (Login & Registrasi)

Sistem menyediakan fungsionalitas bagi pengguna baru untuk membuat akun (*register*) dengan menyimpan *username*, *password*, dan *nama*. Selain itu, sistem dapat memfasilitasi proses masuk (*login*) dengan mencocokkan *username* dan *password* yang tersimpan di basis data. Setelah *login*, sistem akan mengarahkan pengguna ke halaman yang sesuai berdasarkan peran (*role*) mereka, yakni sebagai admin atau pelanggan.

3.1.1.2. Fitur Manajemen Data Sistem (Admin)

Aplikasi menyediakan *dashboard* khusus bagi Admin untuk melakukan operasi CRUD (*Create, Read, Update, Delete*) terhadap data-data krusial bioskop. Admin dapat mengelola:

1. Data Film: Menambah film baru beserta informasi judul, genre, rating usia, sinopsis, poster, dan durasinya.
2. Data Studio: Mengatur ruang penayangan (tipe Reguler atau Premiere) beserta kapasitas kursinya.
3. Data Jadwal Tayang: Menjadwalkan suatu film pada studio, hari, dan jam tertentu beserta penetapan harga tiket dasarnya.
4. Data Promo: Menambahkan kode promo diskon yang dapat diklaim, baik berupa potongan harga tetap (*nominal*) maupun persentase.

3.1.1.3. Fitur Katalog Film dan Jadwal (Pelanggan)

Sistem dapat menampilkan daftar film yang tersedia ke antarmuka pengguna. Pelanggan dapat melihat rincian film (judul, genre, durasi, dll.) serta melihat ketersediaan jadwal penayangan di masing-masing studio berdasarkan hari dan jam tayang.

3.1.1.4. Fitur Pemesanan dan Pemilihan Kursi (Pelanggan)

Sistem memungkinkan pelanggan yang telah memilih jadwal tayang untuk memilih kursi spesifik secara virtual (contoh: A1, B2). Sistem juga harus mampu memvalidasi kursi yang dipilih, di mana kursi yang telah dipesan oleh pelanggan lain dalam jadwal tayang yang sama akan dikunci (*unavailable*) agar tidak terjadi pemesanan ganda.

3.1.1.5. Fitur Transaksi dan Penggunaan Promo (Pelanggan)

Setelah mengkonfirmasi jumlah dan lokasi kursi, sistem akan mengkalkulasi total harga pesanan. Pada fitur ini, pelanggan dapat memasukkan kode promo. Sistem akan memvalidasi *logical connection* ke data promo dan secara otomatis memberikan potongan harga pada total_harga sebelum akhirnya mencatat seluruh data pesanan ke dalam basis data sebagai riwayat transaksi pemesanan tiket.

3.1.2 Kebutuhan Non-Fungsional

Kebutuhan non-fungsional menjelaskan spesifikasi teknis dari perangkat keras dan lunak yang digunakan selama tahap pengembangan, pengujian, dan implementasi aplikasi.

3.1.2.1. Spesifikasi Perangkat Lunak

Sistem ini dibangun dan diujikan dengan spesifikasi perangkat lunak (*software*) sebagai berikut:

1. Sistem Operasi: Windows 10/11 / Linux / macOS (Agnostik)
2. Bahasa Pemrograman: Java (JDK versi 11 atau yang lebih baru)
3. Database Management System: MySQL (menggunakan *tool* seperti XAMPP atau MySQL Workbench)
4. Web / Application Server: Apache Tomcat
5. Library/Teknologi Tambahan: Java Servlet API, JavaServer Pages (JSP), Java Database Connectivity (JDBC) *Driver* untuk MySQL.

3.1.2.2. Spesifikasi Perangkat Keras

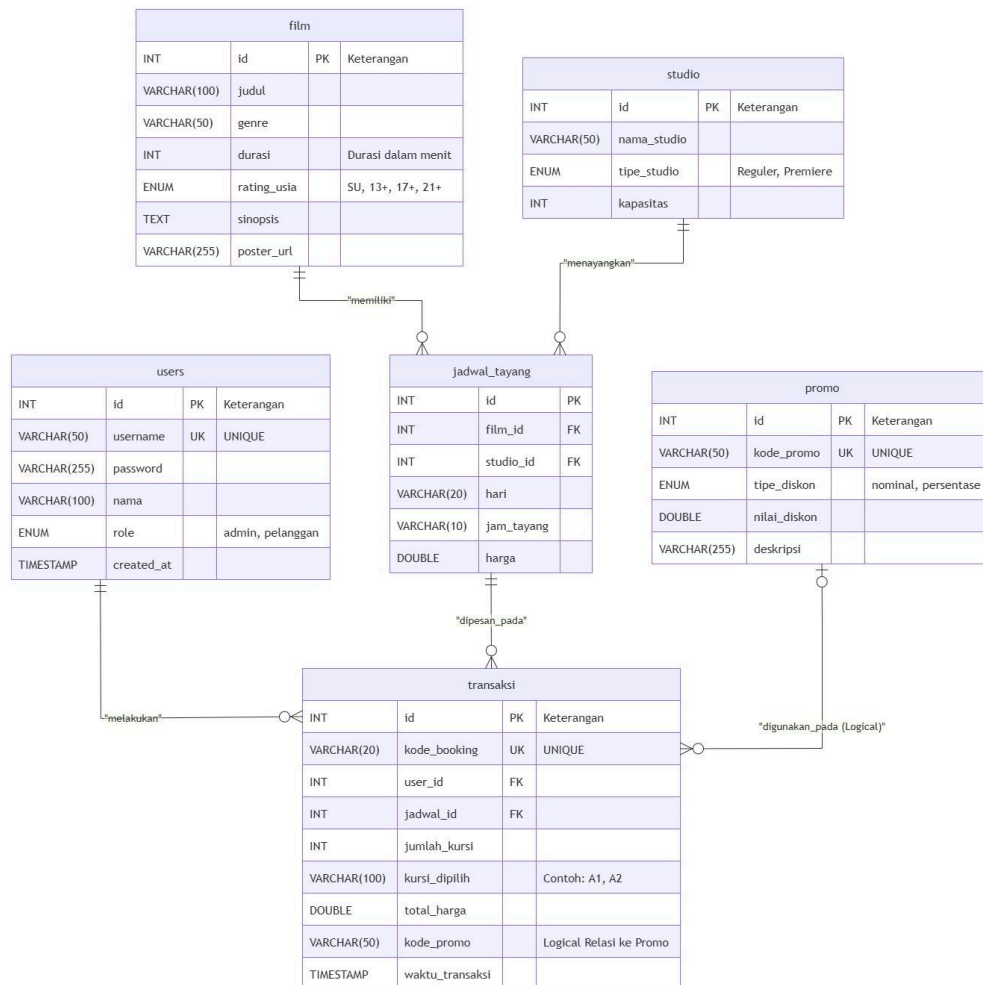
Sistem ini dapat berjalan dan dikembangkan dengan optimal menggunakan perangkat keras (*hardware*) dengan spesifikasi minimum berikut:

1. Prosesor: Intel Core i3 / AMD Ryzen 3 atau yang setara
2. Memori (RAM): Minimal 4GB (Direkomendasikan 8GB untuk kelancaran eksekusi IDE dan Server)

3. Penyimpanan (Storage): Tersedia minimal 2 GB ruang penyimpanan kosong (SSD direkomendasikan)
4. Layar: Resolusi layar minimum 1366 x 768 piksel untuk kenyamanan antarmuka pengguna.

3.2 Implementasi Basis Data

Menampilkan dan menjelaskan diagram ERD



Gambar 1 ERD Sistem Movix

Berdasarkan rancangan ERD di atas, basis data sistem pemesanan tiket bioskop MoviX terdiri dari 6 (enam) entitas utama yang saling berelasi untuk mendukung proses pemesanan tiket. Berikut adalah penjelasan untuk masing-masing entitas:

Entitas users Menyimpan data pengguna yang terdaftar pada sistem. Entitas ini menyimpan informasi *login* (username dan password) serta identitas peran (*role*) sebagai admin atau pelanggan. Entitas ini berelasi dengan entitas

transaksi (relasi 1-ke-N) karena satu pengguna dapat melakukan banyak transaksi pemesanan tiket.

Entitas film Menyimpan master data film yang akan atau sedang ditayangkan, mencakup atribut informasi detail film seperti judul, genre, durasi, rating usia, hingga sinopsis. Entitas ini berelasi dengan entitas jadwal_tayang (relasi 1-ke-N).

Entitas studio Menyimpan data ruangan bioskop beserta kapasitas tempat duduk dan tipe studionya (Reguler atau Premiere). Entitas ini berelasi dengan entitas jadwal_tayang (relasi 1-ke-N).

Entitas jadwal_tayang Merupakan entitas transaksional yang menghubungkan film dan studio pada hari dan jam tayang tertentu, beserta penetapan harga tiket. Entitas ini memiliki relasi (1-ke-N) terhadap entitas transaksi, di mana satu jadwal tayang dapat dipesan pada banyak transaksi.

Entitas promo Menyimpan data kode diskon yang dapat digunakan oleh pelanggan. Entitas ini menyimpan tipe diskon (nominal atau persentase) beserta besaran nilainya. Entitas ini memiliki relasi logikal ke entitas transaksi untuk mencatat kode promo apa yang diterapkan saat pembayaran.

Entitas transaksi Merupakan entitas pencatatan utama untuk setiap pemesanan tiket. Entitas ini menyimpan *foreign key* dari users dan jadwal_tayang, serta mencatat detail pemesanan seperti jumlah kursi, nomor kursi yang dipilih (misal: A1, A2), total harga, dan waktu transaksi.

3.2.1 Struktur Basis data

3.2.1.1. Tabel User

Tabel *User* dirancang sebagai berikut;

Tabel 1 Tabel Entitas User

Field	Tipe	Key	Keterangan
id	INT	Primary Key	Auto Increment
username	VARCHAR(50)	Unique Key	UNIQUE
password	VARCHAR(255)	-	

nama	VARCHAR(100)	-	
role	ENUM	-	'admin', 'pelanggan'
created_at	TIMESTAMP	-	

3.2.1.2. Tabel film

Field	Tipe	Key	Keterangan
id	INT	Primary Key	Auto Increment
judul	VARCHAR(100)	-	
genre	VARCHAR(50)	-	
durasi	INT	-	Durasi dalam menit
rating_usia	ENUM	-	'SU', '13+', '17+', '21+'
sinopsis	TEXT	-	
poster_url	VARCHAR(255)	-	Path atau URL poster film

3.2.1.3. Tabel studio

Field	Tipe	Key	Keterangan
id	INT	Primary Key	Auto Increment
nama_studio	VARCHAR(50)	-	
tipe_studio	ENUM	-	'Reguler', 'Premiere'
kapasitas	INT	-	Jumlah maksimal kursi

3.2.1.4. Tabel jadwal_tayang

Field	Tipe	Key	Keterangan
id	INT	Primary Key	Auto Increment
film_id	INT	Foreign Key	Relasi ke film.id
studio_id	INT	Foreign Key	Relasi ke studio.id
hari	VARCHAR(20)	-	
jam_tayang	VARCHAR(10)	-	

harga	DOUBLE	-	Harga dasar per tiket
-------	--------	---	-----------------------

3.2.1.5. Tabel promo

Field	Tipe	Key	Keterangan
id	INT	Primary Key	Auto Increment
kode_promo	VARCHAR(50)	Unique Key	UNIQUE
tipe_diskon	ENUM	-	'nominal', 'persentase'
nilai_diskon	DOUBLE	-	Besaran diskon
deskripsi	VARCHAR(255)	-	

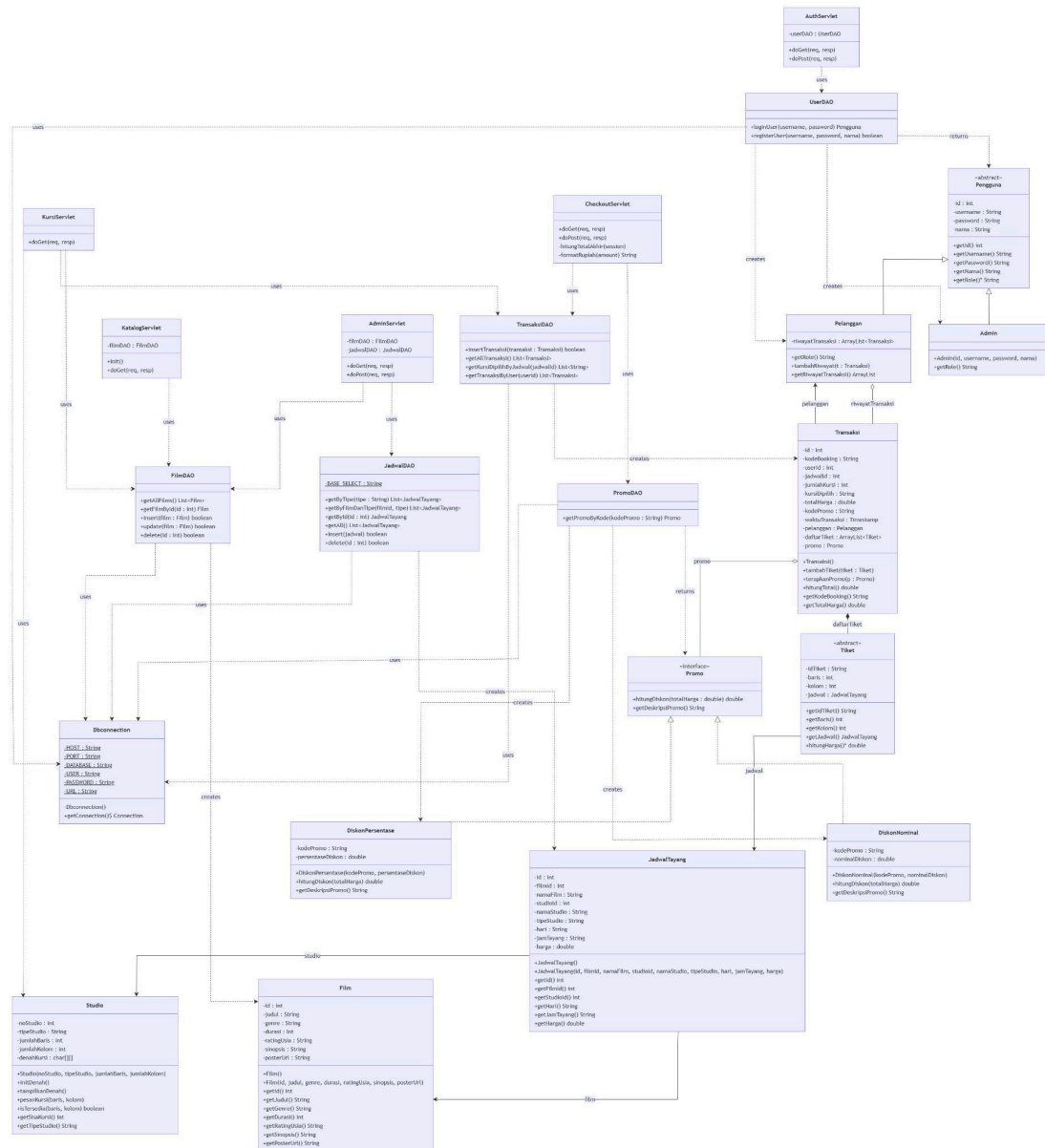
3.2.1.6. Tabel transaksi

Field	Tipe	Key	Keterangan
id	INT	Primary Key	Auto Increment
kode_booking	VARCHAR(20)	Unique Key	UNIQUE

user_id	INT	Foreign Key	Relasi ke users.id
jadwal_id	INT	Foreign Key	Relasi ke jadwal_tayang.id
jumlah_kursi	INT	-	
kursi_dipilih	VARCHAR(100)	-	Contoh: "A1, A2"
total_harga	DOUBLE	-	Harga akhir setelah promo
kode_promo	VARCHAR(50)	-	Logical Relasi ke promo
waktu_transaksi	TIMESTAMP	-	

3.3 Implementasi UML

3.3.1 Class Diagram



3.3.2 Deskripsi Kelas

Berdasarkan *Class Diagram* yang dirancang, arsitektur sistem dibangun dengan pendekatan pola *Model-View-Controller* (MVC) dan *Data Access Object* (DAO). Kelas-kelas dalam sistem dikelompokkan berdasarkan tanggung jawabnya untuk menjaga modularitas dan mempermudah pemeliharaan kode.

3.3.2.1. Kelas Entitas Pengguna (Inheritance)

- **Kelas Pengguna (*Abstract*):** Merupakan kelas induk (*superclass*) yang merepresentasikan entitas pengguna secara umum. Kelas ini membungkus (*encapsulation*) atribut dasar seperti id, username, password, dan nama beserta *method getter/setter*-nya.

- Kelas Admin: Merupakan *subclass* turunan dari Pengguna yang merepresentasikan aktor dengan hak akses manajemen sistem.
- Kelas Pelanggan: Merupakan *subclass* turunan dari Pengguna yang merepresentasikan aktor pembeli tiket. Kelas ini memiliki agregasi terhadap daftar Transaksi untuk melacak riwayat pesanan.

3.3.2.2. Kelas Logika Promo (Polymorphism & Abstraction)

- Interface Promo: Sebuah antarmuka (*interface*) yang mendefinisikan kontrak fungsi hitungDiskon() dan getDeskripsiPromo().
- Kelas DiskonPersentase: Kelas yang mengimplementasikan *interface* Promo dengan logika perhitungan diskon berdasarkan persentase (contoh: pemotongan 10% atau 20% dari harga tiket).
- Kelas DiskonNominal: Kelas yang mengimplementasikan *interface* Promo dengan logika perhitungan diskon berupa potongan harga tetap (contoh: potongan Rp15.000).

3.3.2.3. Kelas Entitas Data Utama (POJO)

- Kelas Film: Menyimpan status informasi spesifik sebuah film, seperti judul, genre, durasi, ratingUsia, sinopsis, dan posterUrl.
- Kelas Studio: Menyimpan informasi ruang tayang meliputi namaStudio, tipeStudio, dan fungsi manajemen *array* dua dimensi untuk tampilkanKursi().
- Kelas JadwalTayang: Bertindak sebagai penghubung antara objek Film dan Studio, menyimpan hari, jamTayang, dan penetapan harga dasar tiket.
- Kelas Transaksi dan Tiket: Menampung logika pemesanan, mencatat kursi yang dipilih, memanggil metode Promo, dan menghitung total pembayaran akhir pengguna.

3.3.2.4. Kelas Data Access Object (DAO)

- Kelas UserDAO: Menangani operasi ke basis data khusus untuk tabel users, seperti validasi login dan registrasi.
- Kelas FilmDAO, JadwalDAO, PromoDAO, TransaksiDAO: Menangani seluruh operasi CRUD (*Create, Read, Update, Delete*) dari dan ke basis data MySQL sesuai dengan entitasnya masing-masing.

3.3.2.5. Kelas Controller (Servlet)

- Kelas AuthServicelet: Menerima *request* form login/registrasi dari halaman *View* (JSP/HTML) dan meneruskannya ke UserDao.
- Kelas AdminServlet: Mengatur lalu lintas data untuk halaman *dashboard* manajemen Admin.
- Kelas KatalogServlet, KursiServlet, CheckoutServlet: Memproses alur pemesanan pelanggan dari mulai melihat daftar film, memilih kursi kosong, hingga menyelesaikan pembayaran tiket.

3.3.2.6. Kelas Utilitas Koneksi

- Kelas DbConnection: Bertanggung jawab penuh untuk membuat dan memutus aliran koneksi dengan basis data MySQL menggunakan parameter spesifik seperti *host*, *port*, *username*, dan *password*. Objek koneksi dari kelas ini (melalui getConnection()) akan dipanggil oleh seluruh kelas DAO.

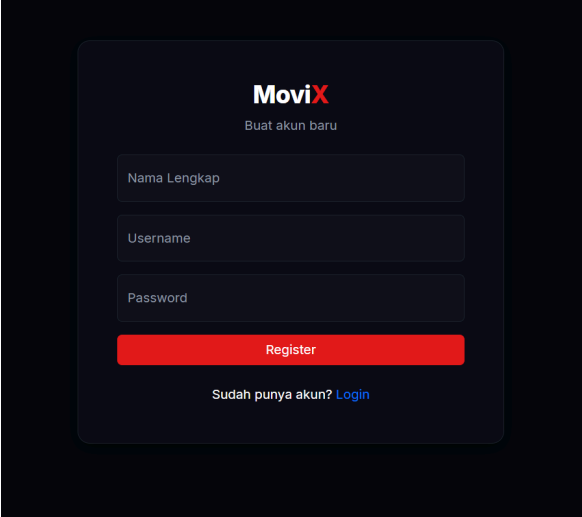
3.4 Implementasi Antarmuka Pengguna (*User Interface*)

Antarmuka pengguna pada sistem MoviX dirancang agar interaktif, responsif, dan mudah digunakan (user-friendly) oleh pengguna. Implementasi *View* ini menggunakan teknologi HTML, CSS, dan JSP (*JavaServer Pages*) yang terhubung langsung dengan *Controller* (Servlet).

3.4.1 Halaman *Login*

Halaman ini merupakan pintu masuk utama ke dalam sistem.

- Halaman Registrasi: Menampilkan form bagi pengguna baru untuk mendaftarkan akun dengan mengisi username, password, dan nama lengkap.

The image shows a registration form for 'MoviX' on a dark background. The form is a light gray rounded rectangle. At the top, it says 'MoviX' in white and red, followed by 'Buat akun baru' in small white text. There are three input fields: 'Nama Lengkap', 'Username', and 'Password'. Below these is a red 'Register' button. At the bottom, it says 'Sudah punya akun? Login' with 'Login' as a blue link.

- Halaman Login: Menampilkan form input untuk username dan password. Sistem akan memvalidasi data yang dimasukkan. Jika sesuai, sistem akan mengarahkan pengguna ke halaman Dashboard Admin atau Halaman Utama Pelanggan berdasarkan role pengguna tersebut.

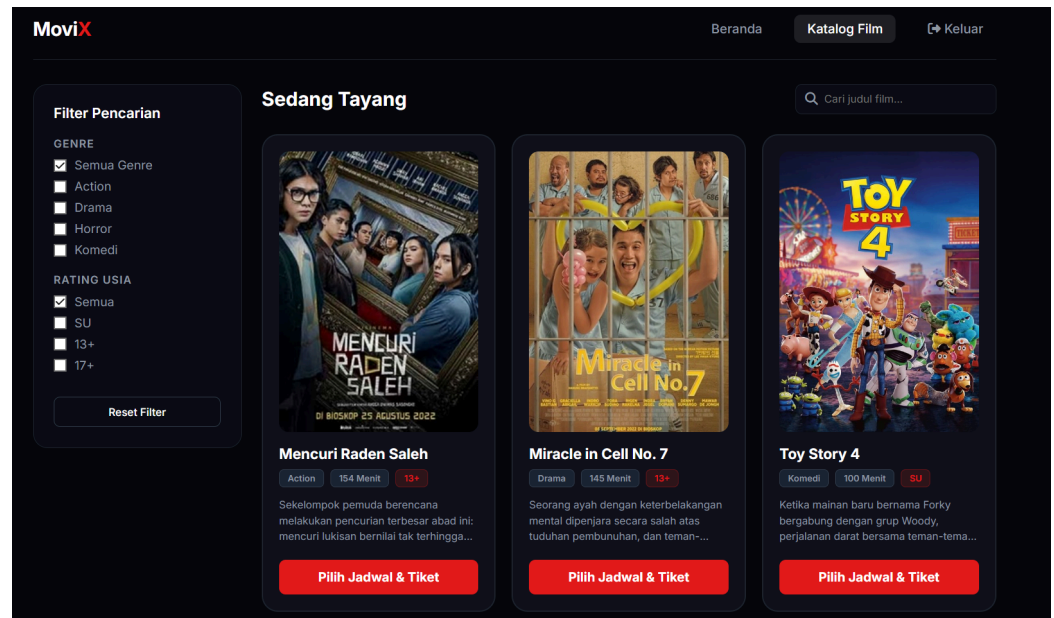
3.4.2 Halaman Dashboard Admin

Halaman ini hanya dapat diakses oleh pengguna dengan *role* Admin. Antarmuka ini menampilkan menu navigasi untuk mengelola data inti (*Master Data*) bioskop. Pada halaman ini, Admin dapat melihat tabel daftar Film, Studio, Jadwal Tayang, dan Promo. Terdapat juga tombol aksi untuk menambah (Create), mengubah (Update), dan menghapus (Delete) data pada masing-masing tabel manajemen tersebut.

ID	Judul Film	Studio	Waktu	Harga	Aksi
13	A Quiet Place	Studio 1	Jumat, 22:00	Rp 45000.0	Edit Hapus
10	Interstellar	Studio 1	Kamis, 15:00	Rp 35000.0	Edit Hapus
15	Pengabdi Setan 2: Communion	Studio 3	Kamis, 19:30	Rp 35000.0	Edit Hapus
16	Oppenheimer	Studio 1	Minggu, 10:00	Rp 45000.0	Edit Hapus
12	Spider-Man: No Way Home	Studio 3	Minggu, 14:00	Rp 45000.0	Edit Hapus
6	Dua Garis Biru	Studio 1	Rabu, 11:00	Rp 35000.0	Edit Hapus
7	Dua Garis Biru	Studio 1	Rabu, 14:30	Rp 35000.0	Edit Hapus
19	Mencuri Raden Saleh	Studio 1	Sabtu, 16:00	Rp 45000.0	Edit Hapus
9	Parasite	Studio 3	Selasa, 13:00	Rp 35000.0	Edit Hapus

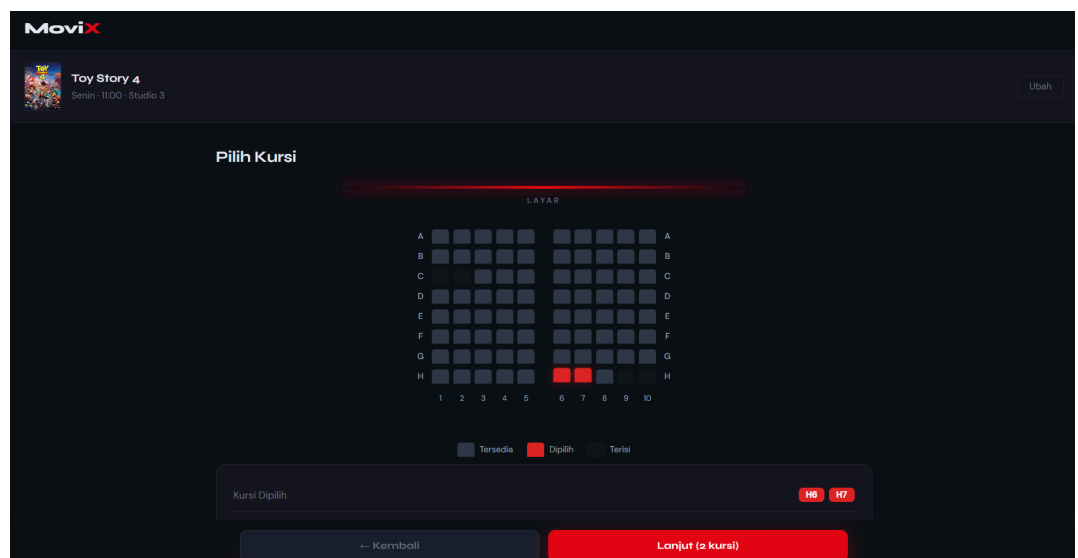
3.4.3 Halaman Katalog Film (Beranda Pelanggan)

Halaman ini merupakan halaman utama yang dilihat oleh Pelanggan setelah berhasil *login*. Antarmuka menampilkan daftar film yang sedang tayang (berupa *grid* atau *list*) lengkap dengan poster film, judul, genre, dan rating usia. Setiap kartu film memiliki tombol interaktif (misalnya tombol "Lihat Jadwal" atau "Pesan Tiket") yang akan mengarahkan pelanggan ke halaman pemilihan jadwal.



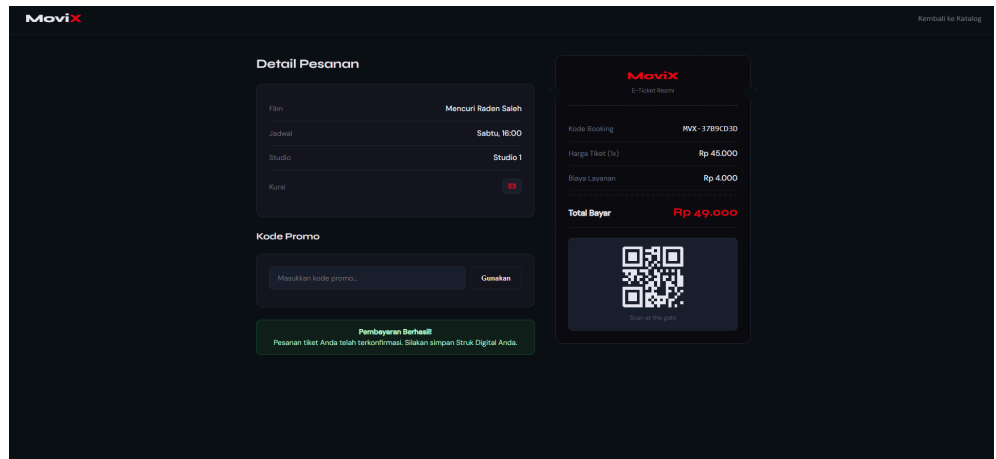
3.4.4 Halaman Pemilihan Kursi

Setelah pelanggan memilih jadwal tayang dan studio, pelanggan akan diarahkan ke halaman ini. Antarmuka menampilkan tata letak (*layout*) kursi di dalam studio. Kursi ditampilkan dengan indikator warna atau ikon yang berbeda untuk membedakan antara kursi yang masih "Tersedia" (*Available*), kursi yang "Sudah Dipesan" (*Booked*), dan kursi yang "Sedang Dipilih" oleh pelanggan.



3.4.5 Halaman Checkout dan Pembayaran

Halaman ini menampilkan ringkasan pesanan pelanggan, meliputi judul film, jadwal, lokasi studio, dan nomor kursi yang dipilih. Pada halaman ini terdapat *form input* bagi pelanggan untuk memasukkan "Kode Promo". Sistem akan secara dinamis memperbarui rincian harga (total harga awal, jumlah diskon, dan total yang harus dibayar) sebelum pelanggan menekan tombol konfirmasi untuk menyelesaikan transaksi.



4 Pengujian Sistem

4.1 Tujuan Pengujian

Memastikan seluruh modul dari aplikasi MoviX (Login, Pendaftaran, CRUD Film, dan Pemesanan) berjalan sesuai dengan rancangan sistem tanpa ada cacat logika (bug).

4.2 Lingkungan Pengujian

4.2.1 Perangkat Lunak Pengujian

Sistem ini diujikan dengan spesifikasi perangkat lunak (*software*) sebagai berikut;

- **Sistem Operasi** : Windows 11 64-bit
- **Bahasa Pemrograman** : Java (JDK 11 atau lebih baru)
- **Database** : MySQL (melalui *stack* XAMPP)
- **Web Server** : Apache Tomcat
- **Web Browser** : Google Chrome / Mozilla Firefox

4.2.2 Perangkat Keras Pengujian

Sistem ini diujikan dengan spesifikasi perangkat Keras sebagai berikut;

- Prosesor : Intel Core I5 14450HX
- Memori : 16 GB DDR5
- GPU : Nvidia RTX 4060
- Penyimpanan : SSD 1 TB

4.3 Metode Pengujian

4.3.1 (Black Box/White Box) Testing

Pengujian fungsionalitas sistem berdasarkan input dan output pada antarmuka pengguna tanpa perlu melihat source code secara mendalam.

4.3.2 Skenario Pengujian

Pengujian dilakukan pada seluruh fitur utama sistem. Skenario pengujian dapat dilihat sebagai berikut

Tabel 2 Identifikasi Pengujian

Kelas Uji	Butir Uji	Identifikasi	Jenis Pengujian	Penguji
<i>Login</i>	Pengujian Fungsi <i>Login</i>	AO-LI01	Black Box	Sava
	Pengujian Kebenaran Fungsi <i>Login</i>	AO-LI02	Black Box	Sava
<i>Logout</i>	Pengujian Fungsi <i>logout</i>	AO-LO01	Black Box	Axel
Transaksi	Pengujian fungsionalitas pemilihan kursi secara dinamis	AO-TRX01	Black Box	Daffa
Transaksi	Pengujian pencegahan pemilihan kursi yang sudah dipesan	AO-TRX02	Black Box	Daffa
Promo	Pengujian validasi kode promo dan kalkulasi potongan harga	AO-PRM01	Black Box	Shaquille & Axel
Manajemen Film	Pengujian fungsi Admin dalam menambah data <i>Film</i> baru	AO-FLM01	Black Box	Axel & Ghifari
Pemesanan Akhir	Pengujian penyimpanan data <i>Checkout</i> ke tabel transaksi	AO-CHK01	Black Box	Ghifari & Axel

5 Hasil Uji

Hasil uji berdasarkan skenario yang telah dibuat pada bab sebelumnya.

5.1 Fitur *Login*

5.1.1 Kasus uji

Beberapa kasus uji yang dilakukan sebagai berikut;

Tabel 3 Kasus Uji Fitur Login

Identifikasi	Skenario
AO-LI01	Berhasil <i>Login</i>
AO-LI02	<i>Username</i> Salah
AO-LI02	<i>Password</i> Salah
AO-LI02	<i>Field</i> Kosong

5.1.2 Hasil

Hasil Uji Sebagai Berikut

Tabel 4 Hasil Uji Fitur Login

Identifikasi	<i>Input</i>	Hasil yang diharapkan	Hasil sebenarnya	Status
AO-LI01	<i>Username</i> : admin <i>Password</i> : Admin123	<i>Login</i> Berhasil	Sesuai	Pengujian berhasil
AO-LI02	<i>Username</i> : salah <i>Password</i> : Admin123	Pesan <i>Error</i>	Sesuai	Pengujian berhasil
AO-LI02	<i>Username</i> : admin <i>Password</i> : salah	Pesan <i>Error</i>	Sesuai	Pengujian berhasil
AO-LI02	Kosong	Validasi muncul	Sesuai	Pengujian berhasil

5.2 Fitur *Logout*

5.2.1 Kasus uji

Beberapa kasus uji yang dilakukan sebagai berikut;

Tabel 5 Kasus Uji Fitur Logout

Identifikasi	Skenario
AO-LO01	Pengguna berhasil melakukan <i>logout</i>

5.2.2 Hasil

Hasil Uji Sebagai Berikut

Tabel 6 Hasil Uji Fitur Logout

Identifikasi	Input	Hasil yang diharapkan	Hasil sebenarnya	Status
AO-LO01	Klik tombol/menu "Logout" pada antarmuka sistem	Sesi pengguna (<i>session</i>) dihapus dari sistem dan pengguna diarahkan kembali ke Halaman Login	Sesuai	Pengujian Berhasil

5.3 Fitur Katalog Film Interaktif (admin)

5.3.1 Kasus uji

Beberapa kasus uji yang dilakukan sebagai berikut;

Tabel 5 Kasus Uji Fitur Katalog Film Interaktif

Identifikasi	Skenario
AO-MF01	Admin berhasil menambahkan data film baru
AO-MF02	Admin mengosongkan <i>field</i> wajib saat menambahkan film
AO-MF03	Admin menghapus data film dari sistem

5.3.2 Hasil

Hasil Uji Sebagai Berikut

Tabel 6 Hasil Uji Fitur Katalog Film Interaktif

Identifikasi	Input	Hasil yang diharapkan	Hasil sebenarnya	Status
AO-MF01	Input Judul, Genre, Durasi, Rating, Sinopsis, dan Poster URL valid -> Klik "Simpan"	Data film baru tersimpan di basis data dan langsung tampil di halaman katalog	Sesuai	Pengujian Berhasil
AO-MF02	Mengosongkan isian "Judul" dan "Durasi" ->	Sistem menolak input dan menampilkan	Sesuai	Pengujian Berhasil

	Klik "Simpan"	pesan peringatan (validasi <i>form</i>)		
AO-MF03	Klik tombol/ikon "Hapus" pada salah satu daftar film	Data film beserta relasi jadwal tayangnya terhapus dari basis data dan katalog	Sesuai	Pengujian Berhasil

5.4 Fitur Pemilihan Jadwal dan Studio

5.4.1 Kasus uji

Beberapa kasus uji yang dilakukan sebagai berikut;

Tabel 5 Kasus Uji Fitur Pemilihan Jadwal dan Studio

Identifikasi	Skenario
AO-JS01	Pengguna memilih salah satu film
AO-JS02	Pengguna memilih jadwal studio reguler
AO-JS03	Pengguna memilih jadwal studio premier

5.4.2 Hasil

Hasil Uji Sebagai Berikut

Tabel 6 Hasil Uji Fitur Pemilihan Jadwal dan Studio

Identifikasi	Input	Hasil yang diharapkan	Hasil sebenarnya	Status
AO-JS01	Menekan salah satu film yang ada di home	Pindah ke page pemilihan studio/jenis tiket	Sesuai	Pengujian berhasil
AO-JS02	Menekan tombol pilih tiket/studio reguler	Memperlihatkan jadwal film yang dipilih pada studio-studio reguler	Sesuai	Pengujian berhasil
AO-JS03	Menekan tombol pilih tiket/studio premier	Memperlihatkan jadwal film yang dipilih pada studio-studio premier	Sesuai	Pengujian berhasil

5.5 Fitur Pemilihan Kursi Dinamis

5.5.1 Kasus uji

Beberapa kasus uji yang dilakukan sebagai berikut;

Tabel 5 Kasus Uji Fitur Pemilihan Kursi Dinamis

Identifikasi	Skenario
AO-KD01	Menampilkan matriks kursi sesuai dengan total kapasitas dan tata letak <i>Studio</i>
AO-KD02	Sistem memberikan indikator visual yang berbeda untuk kursi yang <i>Available</i> (Tersedia), <i>Selected</i> (Dipilih), dan <i>Booked</i> (Sudah Terpesan)
AO-KD03	Mengklik kursi yang berstatus <i>Available</i>
AO-KD04	Mengklik kursi yang berstatus <i>Booked</i> (Sudah dipesan di transaksi lain)
AO-KD05	Membatalkan pilihan dengan mengklik kembali kursi yang berstatus <i>Selected</i>

5.5.2 Hasil

Hasil Uji Sebagai Berikut

Tabel 6 Hasil Uji Fitur Pemilihan Kursi Dinamis

Identifikasi	Input	Hasil yang diharapkan	Hasil sebenarnya	Status
AO-KD01	Membuka halaman pemilihan kursi untuk Studio Reguler (Kapasitas 50)	Tampil kursi dalam format grid (misal: 5 baris x 10 kolom)	Sesuai	Pengujian berhasil
AO-KD02	Melihat tampilan halaman kursi	Kursi <i>Available</i> berwarna putih/hijau, kursi <i>Booked</i> berwarna abu-abu/merah	Sesuai	Pengujian berhasil

AO-KD03	Klik pada elemen kursi 'C3' yang masih kosong	Kursi 'C3' berubah status menjadi <i>Selected</i> , id kursi masuk ke daftar pesanan sementara	Sesuai	Pengujian berhasil
AO-KD04	Klik pada elemen kursi 'A1' yang berstatus <i>Booked</i>	Tidak terjadi aksi apa-apa (<i>disabled</i>), kursi tidak masuk ke daftar pesanan	Sesuai	Pengujian berhasil
AO-KD05	Klik kembali elemen kursi 'C3' yang sedang berstatus <i>Selected</i>	Kursi kembali berstatus <i>Available</i> , id 'C3' dihapus dari daftar pesanan sementara	Sesuai	Pengujian berhasil

5.6 Fitur Sistem Checkout dan Kode Promo

5.6.1 Kasus uji

Beberapa kasus uji yang dilakukan sebagai berikut;

Tabel 5 Kasus Uji Fitur Sistem Checkout dan Kode Promo

Identifikasi	Skenario
TC-CHK-01	Membuka halaman <i>checkout</i> dan memverifikasi kesesuaian data pesanan (Judul Film, Jadwal, Studio, Kursi) yang dibawa dari halaman sebelumnya.
TC-CHK-02	Memasukkan kode promo yang valid, menekan tombol "Gunakan", dan memastikan diskon diterapkan serta total bayar berkurang.
TC-CHK-03	Mengeklik tautan "Hapus" pada promo yang sedang aktif dan memastikan total bayar kembali ke harga normal.
TC-CHK-04	Melakukan verifikasi keakuratan perhitungan matematis "Total Bayar" pada struk berdasarkan harga tiket,

	jumlah kursi, diskon, dan biaya layanan.
TC-CHK-05	Mengklik tombol "Konfirmasi Pembayaran" dan memastikan sistem memproses transaksi, menampilkan <i>QR Code</i> pembayaran.

5.6.2 Hasil

Hasil Uji Sebagai Berikut

Tabel 6 Hasil Uji Fitur Sistem Checkout dan Kode Promo

Identifikasi	Input	Hasil yang diharapkan	Hasil sebenarnya	Status
TC-CHK-01	Membuka halaman <i>checkout</i> dengan membawa data sesi (film, jadwal, kursi) dari halaman sebelumnya.	Menampilkan Judul Film, Jadwal, Nama Studio, dan Label Kursi secara akurat sesuai dengan pilihan pengguna.	Sesuai	Pengujian berhasil
TC-CHK-02	Memasukkan teks kode promo yang valid ke dalam <i>form</i> dan mengklik tombol "Gunakan"	Menampilkan notifikasi sukses, memunculkan baris "Diskon Promo", dan mengurangi nominal "Total Bayar".	Sesuai	Pengujian berhasil
TC-CHK-03	Mengeklik tautan teks "Hapus" pada deskripsi promo yang sedang aktif.	Menghapus sesi promo, menghilangkan baris diskon dari struk, dan mengembalikan nominal "Total Bayar" ke harga awal.	Sesuai	Pengujian berhasil
TC-CHK-04	Memeriksa rincian	Menampilkan hasil kalkulasi	Sesuai	Pengujian berhasil

	pada kotak struk <i>E-Ticket</i> di sisi kanan layar.	yang tepat sesuai rumus: (Harga Tiket x Jumlah Kursi) - Diskon Promo + Biaya Layanan.		
TC-CHK-05	Mengklik tombol "Konfirmasi Pembayaran".	Memproses pesanan, menampilkan <i>QR Code</i> pembayaran.	Sesuai	Pengujian berhasil

6 Penutup

6.1 Kesimpulan

Aplikasi pemesanan tiket bioskop MoviX telah berhasil dirancang dan dikembangkan dengan menerapkan konsep-konsep utama Pemrograman Berorientasi Objek (OOP), seperti *Inheritance* pada pengelolaan data pengguna, *Polymorphism* pada perhitungan promo, serta *Encapsulation* dalam pengelolaan atribut pada setiap entitas. Selain itu, penerapan arsitektur MVC yang dipadukan dengan DAO dan JDBC membantu memisahkan proses bisnis dari pengelolaan database MySQL, sehingga struktur program menjadi lebih terorganisir dan mudah dikelola. Berdasarkan hasil pengujian *Black-box*, seluruh fitur utama aplikasi dapat berjalan sesuai dengan kebutuhan yang telah ditentukan.

6.2 Saran

Untuk pengembangan sistem ke depan, disarankan untuk:

1. Menambahkan integrasi *payment gateway* (seperti Midtrans atau e-wallet API) untuk otomatisasi verifikasi pembayaran transaksi.
2. Menerapkan enkripsi hashing yang lebih kuat pada penyimpanan password user di database demi meningkatkan aspek keamanan data.
3. Mengembangkan sistem notifikasi otomatis pengiriman kode booking melalui email pelanggan menggunakan JavaMail API.

Daftar Pustaka

Alur, D., Crupi, J., & Malks, D. (2003). *Core J2EE Patterns: Best Practices and Design Strategies* (2nd ed.). Prentice Hall.

Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.

Fowler, M. (2003). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley Professional.

Oracle. (2023). *Java Platform, Standard Edition Documentation*. Diakses dari <https://docs.oracle.com/en/java/>

Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.

Schildt, H. (2018). *Java: The Complete Reference* (11th ed.). McGraw-Hill Education.

Sharan, K. (2018). *Beginning Java 8 Fundamentals: Language Syntax, Arrays, Data Types, Objects, and Regular Expressions*. Apress.

Lampiran

Lampiran A Pembagian Tugas

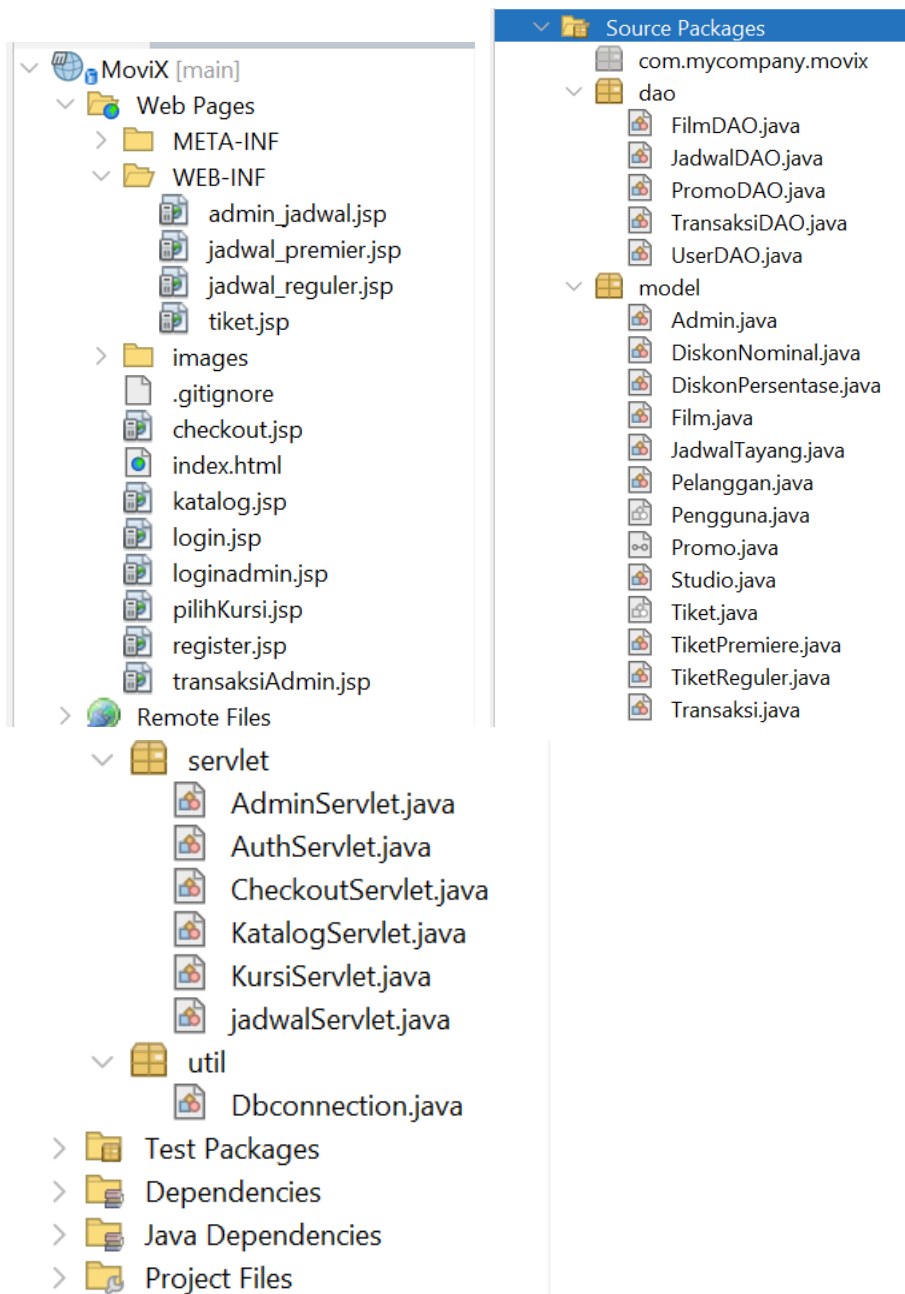
NIM	Nama	Tugas	Kontribusi (100%)	Penyelesaian (%)
103012400020	Ghifari Nurwafi Yoga	1. Membuat UI pemilihan studio dan jadwal tayang 2. Membuat backend awal 3. Membuat UI add, edit, delete admin	20%	100%
103012400107.	Daffa Dzaki Taufiiqul Hakim	1 membuat UI kursi dinamis. 2. Mengupdate backend hingga selesai	20%	100%
103012400043	Sava Arsyia Syandana	1. Membuat UI login user dan admin serta registrasi 2. Membuat backend login dan registrasi	20%	100%
103012400419	Aloysius Axel Adriano	1. Membuat UI home page (list film) 2. Menghubungkan backend 3. Membuat database untuk testing	20%	80%
103012400225	Muhammad Shaquille Shah Alam Murtopo	1. Membuat UI checkout 2. Menambahkan database promo dan diskon	20%	0%

Lampiran B Source Code Penting

Link github:

https://github.com/yogafari793-maker/Tugas-Besar-IF4806-MoviX-Kelompok_7.git

Lampiran C Screenshot Sistem



Lampiran D Manual Instalasi Sistem

1. Kebutuhan Sistem

Sebelum menjalankan aplikasi, pastikan perangkat telah terpasang:

- Java Development Kit (JDK) 17 atau lebih baru
- Apache Tomcat 11
- MySQL Server 8.0
- NetBeans IDE 17 (opsional, untuk pengembangan)
- Git (opsional)

2. Instalasi Database

- Jalankan MySQL
- Buat database baru dengan nama “movix”

- Import file sql

3. Konfigurasi Database

Buka file Dbconnection.java dan samakan seperti berikut.

```
private static final String URL = "jdbc:mysql://localhost:3306/movix";
```

```
private static final String USERNAME = "root";
```

```
private static final String PASSWORD = "password";
```

4. Menjalankan Aplikasi

- Buka proyek MoviX pada NetBeans.
- Pastikan Apache Tomcat telah terhubung ke NetBeans.
- Klik Clean and Build Project.
- Klik Run Project.
- Admin:
 - a. username: admin
 - b. password: admin123
- Pelanggan:
 - a. username: pelanggan
 - b. password: pelanggan123